



گزارش پروژه شماره ۱: جستجوی ناآگاهانه و آگاهانه

لینک گیت‌هاب پروژه: <https://github.com/Elham-ch/AI-Projects>

۱ معرفی مسئله

در این پروژه، مسئله را به صورت یک دنیای شبکه‌ای در نظر گرفتیم. شخصیت اصلی باید تمام قطعات Dragon Glass را که در نقشه با هدف‌ها نمایش داده شده‌اند جمع‌آوری کند. هم‌زمان، چند نوع مانع و خطر در محیط وجود دارد:

- خانه‌های عادی که عبور از آن‌ها هزینه کمی دارد.
- خانه‌های یخی که طبق متن پروژه حرکت را کندتر و پرهزینه‌تر می‌کنند.
- خانه‌های بسته یا خرابه‌ها که عبور از آن‌ها مجاز نیست.
- دشمن متحرک، یعنی Night King، که برخورد با آن در حالت عادی باعث شکست می‌شود.
- سلاح ویژه که در بعضی نقشه‌ها وجود دارد و گرفتن آن می‌تواند برخورد با دشمن را بی‌خطر کند یا حتی دشمن را حذف کند.

هدف ما پیاده‌سازی و مقایسه الگوریتم‌های جستجوی UCS، DLS/IDS، BFS و A^* بود. برای مقایسه، فقط رسیدن به هدف کافی نیست؛ باید هزینه مسیر، تعداد حرکت‌ها، تعداد گره‌های بسط داده‌شده و زمان اجرا را هم در نظر بگیریم. در صورت پروژه برای تابع اکتشافی A^* هم تأکید شده بود که کیفیت آن با تعداد گره‌های بسط داده‌شده سنجیده می‌شود؛ برای مثال در نقشه ساده، کمتر از ۱۰۰ گره بسط داده‌شده امتیاز کامل می‌گیرد.

۲ فرض‌ها و قراردادهای پیاده‌سازی

برای اینکه تحلیل الگوریتم‌ها روشن باشد، فرض‌ها و قراردادهای زیر را در کل پروژه در نظر گرفتیم:

- محیط کاملاً قابل مشاهده و قطعی فرض شده است؛ یعنی نقشه، هدف‌ها، مسیر دشمن و هزینه خانه‌ها از ابتدا معلوم هستند.
- حرکت‌ها فقط در چهار جهت اصلی انجام می‌شوند و حرکت قطری نداریم.
- هزینه هر حرکت برابر هزینه ورود به خانه مقصد است. بنابراین ورود به خانه عادی هزینه ۵ و ورود به خانه یخی هزینه ۱۰۰ دارد.
- خانه R مانع غیرقابل عبور است. اگر حرکت به بیرون نقشه یا روی مانع باشد، آن حرکت به عنوان جانشین معتبر تولید نمی‌شود.
- حالت جستجو فقط موقعیت عامل نیست؛ مجموعه هدف‌های باقی‌مانده، شماره گام یا چرخه دشمن، داشتن سلاح و زنده بودن دشمن هم بخشی از حالت هستند.
- در الگوریتم‌های جستجو، حالت‌هایی که برخورد شکست‌آمیز با دشمن دارند وارد صف یا پشته نمی‌شوند. این کار به این دلیل است که چنین حالت‌هایی مسیر قابل قبول به هدف نیستند.
- در محاسبه فاصله‌های کمکی برای تابع اکتشافی، دشمن و سلاح وارد Dijkstra نمی‌شوند. آن فاصله فقط کوتاه‌ترین هزینه مکانی روی نقشه را می‌دهد و برای تخمین استفاده می‌شود، نه برای اجرای واقعی حرکت‌ها.
- در گزارش، تعداد گره‌های بسط داده‌شده برابر تعداد فراخوانی‌های `get_successors` است، چون هر بار فراخوانی این تابع یعنی الگوریتم یک حالت را واقعاً گسترش داده است.

۳ ساختار کلی پروژه

کد پروژه به دو بخش اصلی تقسیم شده است:

- پوشه env: تعریف محیط، خواندن نقشه، تولید حالت‌های بعدی، محاسبه هزینه، امتیازدهی و نمایش گرافیکی.
- پوشه search: پیاده‌سازی الگوریتم‌های جستجو.
- فایل‌های مهم پروژه عبارت‌اند از:
- env/constants.py: ثابت‌های هزینه و امتیاز.
- env/map.py: خواندن فایل نقشه و تبدیل آن به داده ساخت‌یافته.
- env/domain.py: تعریف کلاس GameState و منطق انتقال بین حالت‌ها.
- env/game.py: اجرای الگوریتم، نمایش نتیجه و محاسبه امتیاز.
- env/instrumentation.py: شمارش گره‌های بسط داده‌شده.
- search/bfs.py: جستجوی سطحی.
- search/dls.py: جستجوی عمق محدود و تکرار عمق.
- search/ucs.py: جستجوی هزینه یکنواخت.
- search/a_star.py: جستجوی آگاهانه با تابع اکتشافی.

۴ خواندن نقشه

در map.py کلاس MapLoader فایل متنی نقشه را می‌خواند. هر نماد در نقشه معنی خاصی دارد:

- A: موقعیت اولیه عامل.
- E: موقعیت اولیه دشمن.
- T: هدف یا قطعه Dragon Glass.
- B: خانه یخی.
- R: مانع غیرقابل عبور.
- W: سلاح.
- G: خانه عادی.

پس از بخش نقشه، یک رشته جهت حرکت دشمن قرار دارد. تابع `_parse_enemy_path` این رشته را به لیستی از مختصات تبدیل می‌کند. این لیست بعداً با استفاده از شماره گام، موقعیت فعلی، قبلی و بعدی دشمن را مشخص می‌کند.

۵ تولید جانشین‌ها

وقتی یک الگوریتم جستجو یک حالت را انتخاب می‌کند، باید بداند از آن حالت چه حرکت‌هایی ممکن است. تابع `get_successors` همین کار را انجام می‌دهد. این تابع چهار جهت U، D، L و R را یکی یکی امتحان می‌کند. برای هر حرکت، ابتدا خانه مقصد حساب می‌شود. اگر مقصد بیرون نقشه باشد یا روی مانع R قرار بگیرد، آن حرکت کنار گذاشته می‌شود. اگر حرکت معتبر باشد، یک حالت جدید ساخته می‌شود. در این حالت جدید:

- اگر عامل روی یک هدف رفته باشد، آن هدف از مجموعه هدف‌ها حذف می‌شود.
- اگر عامل به خانه سلاح برسد، مقدار `has_weapon` درست می‌شود.
- اگر عامل با داشتن سلاح به موقعیت فعلی یا بعدی دشمن برسد، دشمن حذف می‌شود.
- مقدار `step` یکی زیاد می‌شود تا حرکت دشمن در گام بعدی درست محاسبه شود.

در پایان، هزینه همان حرکت هم محاسبه می‌شود. هزینه رفتن به خانه عادی 5 و هزینه رفتن به خانه یخی 100 است. تابع `_compute_transition_cost` همین مقدار را برمی‌گرداند. اگر عامل بدون سلاح وارد حالت برخورد با دشمن شود، جریمه برخورد هم به هزینه اضافه می‌شود. البته در پیاده‌سازی الگوریتم‌ها، حالت‌های برخوردی معمولاً وارد صف جستجو نمی‌شوند، چون مسیر شکست هستند.

۶ هدف و برخورد

تابع `is_goal_state` زمانی درست است که هیچ هدفی باقی نمانده باشد و حالت برخوردی نباشد:

```
1 len(self._targets) == 0 and not self.is_collision_state()
```

تابع `is_collision_state` بررسی می‌کند که موقعیت عامل با موقعیت فعلی یا قبلی دشمن یکی شده است یا نه. دلیل بررسی موقعیت قبلی دشمن این است که جابه‌جایی هم‌زمان عامل و دشمن می‌تواند حالت عبور از روی هم را ایجاد کند. اگر عامل سلاح داشته باشد، برخورد شکست محسوب نمی‌شود.

۷ شمارش گره‌های بسط داده‌شده

در `instrumentation.py` کلاس `CountingGameState` دور یک حالت عادی پیچیده می‌شود و هر بار که `get_successors` فراخوانی شود، شمارنده یکی زیاد می‌شود. بنابراین تعداد گره‌های بسط داده‌شده در واقع تعداد حالت‌هایی است که الگوریتم از آن‌ها جانشین تولید کرده است. این دقیقاً همان معیاری است که در صورت پروژه برای ارزیابی تابع اکتشافی مهم دانسته شده است.

۸ الگوی مشترک الگوریتم‌های جستجو

برای اینکه توضیح الگوریتم‌ها منظم‌تر باشد، ابتدا ایده مشترک آن‌ها را توضیح می‌دهیم. در این بخش از مقاله `Red Blob Games` درباره A^* هم کمک گرفتیم، چون در آن `BFS`، `UCS` و A^* با یک چارچوب مشترک توضیح داده شده‌اند [۱]. این منبع فقط برای توضیح ایده عمومی جستجوی گرافی، صف اولویت و رابطه $g(n) + h(n)$ استفاده شده است. تابع اکتشافی چندهدفه، استفاده از `MST`، پایین‌کران سه‌تایی و منطق مربوط به سلاح را بر اساس نیازهای همین پروژه طراحی کردیم. ایده اصلی این است که این الگوریتم‌ها اعضای یک خانواده هستند و همگی یک مرز جستجو یا `frontier` دارند. الگوریتم در هر مرحله یک حالت را از `frontier` برمی‌دارد، آن را بسط می‌دهد، و جانشین‌های مناسب را دوباره وارد `frontier` می‌کند. در همه پیاده‌سازی‌ها، جدول `came_from` برای بازسازی مسیر استفاده می‌شود. اگر برای یک حالت `s` داشته باشیم:

```
1 came_from[s] = (previous_state, action)
```

یعنی با انجام `action` از `previous_state` به `s` رسیده‌ایم. بعد از رسیدن به هدف، مسیر از هدف به عقب دنبال می‌شود و سپس معکوس می‌گردد.

تفاوت اصلی الگوریتم‌ها در نوع `frontier` و معیار انتخاب حالت بعدی است:

- در `BFS`، `frontier` یک صف معمولی است و حالت‌ها بر اساس تعداد حرکت بررسی می‌شوند.
- در `UCS`، `frontier` یک صف اولویت است و اولویت برابر هزینه واقعی طی شده است.
- در A^* ، `frontier` باز هم صف اولویت است، اما اولویت برابر جمع هزینه واقعی طی شده و تخمین هزینه باقی‌مانده است.

همچنین در همه الگوریتم‌ها شرط هدف هنگام خارج کردن حالت از `frontier` بررسی می‌شود، نه هنگام اضافه کردن حالت به `frontier`. این قرارداد مخصوصاً برای `UCS` و A^* مهم است، چون ممکن است یک حالت ابتدا با هزینه بیشتر دیده شود و بعداً مسیر ارزان‌تری برای همان حالت پیدا شود.

۹ الگوریتم BFS

در `bfs.py` پیاده‌سازی `BFS` خیلی مستقیم است. از جستجوی سطحی یا `Breadth-First Search` استفاده شده است. هدف `BFS` این است که حالت‌ها را بر اساس تعداد حرکت بررسی کند: اول حالت اولیه، بعد همه حالت‌هایی که با یک حرکت قابل رسیدن هستند، بعد همه حالت‌های دو حرکتی و همین‌طور ادامه می‌دهد. برای این رفتار از `deque` استفاده کردیم، چون می‌توانیم از ابتدای آن با `popleft` برداریم و به انتهای آن با `append` اضافه کنیم. این دقیقاً رفتار صف `FIFO` است:

```
1 frontier = deque([initial_state])
2 came_from = {initial_state: (None, None)}
```

متغیر `frontier` مرز جستجو است؛ یعنی حالت‌هایی که دیده شده‌اند اما هنوز بسط داده نشده‌اند. در ابتدا فقط حالت اولیه در این صف قرار دارد. دیکشنری `came_from` دو نقش دارد. نقش اول آن این است که مشخص کند هر حالت از کدام حالت قبلی و با کدام عمل به دست آمده است. نقش دوم آن این است که مانند مجموعه `visited` عمل کند و نگذارد یک حالت دوباره وارد صف شود. مقدار `(None, None)` برای حالت اولیه یعنی قبل از آن هیچ حالتی و هیچ عملی وجود نداشته است. حلقه اصلی تا وقتی ادامه دارد که `frontier` خالی نشده باشد. در هر مرحله، قدیمی‌ترین حالت دیده‌شده از ابتدای صف برداشته می‌شود:

```
1 current_state = frontier.popleft()
```

چون حالت‌ها به ترتیب ورود از صف خارج می‌شوند، هر حالتی که زودتر و با عمق کمتر دیده شده باشد زودتر بسط داده می‌شود. سپس بررسی می‌کنیم آیا این حالت هدف است یا نه. اگر هدف باشد، آن را در `goal_state` نگه می‌داریم و جستجو را متوقف می‌کنیم:

```
1 if current_state.is_goal_state():
2     goal_state = current_state
3     break
```

اگر حالت هدف نباشد، جانشین‌های آن با `get_successors` ساخته می‌شوند:

```
1 for action, _, next_state in current_state.get_successors():
2     ...
```

خروجی `get_successors` سه بخش دارد: عمل، هزینه حرکت و حالت بعدی. در BFS هزینه حرکت را با `_` نادیده می‌گیریم، چون BFS بر اساس تعداد حرکت تصمیم می‌گیرد، نه بر اساس هزینه واقعی مسیر. سپس فقط وقتی حالت بعدی را اضافه می‌کنیم که دو شرط برقرار باشد: قبلاً دیده نشده باشد و حالت برخورد شکست‌آمیز با دشمن نباشد.

```
1 if next_state not in came_from and not next_state.is_collision_state():
2     frontier.append(next_state)
3     came_from[next_state] = (current_state, action)
```

شرط اول از حلقه‌های بی‌پایان جلوگیری می‌کند؛ مثلاً عامل نباید بی‌دلیل بین دو خانه رفت‌وبرگشت کند و هر بار همان وضعیت را دوباره وارد صف کند. شرط دوم به این دلیل است که حالت برخوردی بدون سلاح مسیر قابل قبول به جواب نیست، پس نگه داشتن آن فقط تعداد بسط‌ها را زیاد می‌کند. بعد از اضافه کردن حالت به صف، در `came_from` ثبت می‌کنیم که این حالت از `current_state` و با عمل `action` به دست آمده است.

در `came_from` در پایان برای بازسازی مسیر استفاده می‌شود. اگر هیچ هدفی پیدا نشده باشد، تابع لیست خالی برمی‌گرداند. در غیر این صورت، از `goal_state` شروع می‌کنیم، با کمک `came_from` به عقب برمی‌گردیم و عمل‌ها را در لیست `path` می‌گذاریم. چون این مسیر از هدف به ابتدا ساخته شده است، در پایان آن را معکوس می‌کنیم:

```
1 path.reverse()
2 return path
```

در این پروژه، یک حالت فقط مختصات عامل نیست. مجموعه هدف‌های باقی‌مانده، چرخه حرکت دشمن، داشتن یا نداشتن سلاح و زنده بودن دشمن هم بخشی از حالت هستند. به همین دلیل ممکن است عامل در دو زمان مختلف روی یک خانه باشد، اما آن دو وضعیت از نظر جستجو یکسان نباشند.

BFS اگر هزینه همه حرکت‌ها برابر باشد، مسیر بهینه از نظر تعداد حرکت را پیدا می‌کند. اما در این پروژه هزینه ورود به خانه عادی 5 و هزینه ورود به خانه یخی 100 است؛ پس مسیر کم‌حرکت‌تر لزوماً کم‌هزینه‌تر نیست. برای نمونه در `easy-no-weapon`، BFS مسیر ۱۸ حرکتی با هزینه 280 پیدا می‌کند، در حالی که UCS و `A*` مسیر ۲۲ حرکتی با هزینه 110 می‌یابند.

۱۰ الگوریتم DLS/IDS

در `dls.py` ابتدا جستجوی عمق محدود یا Depth-Limited Search پیاده‌سازی شده است. این الگوریتم مثل DFS در عمق جلو می‌رود، اما اجازه ندارد از یک عمق مشخص بیشتر پایین برود. در کد ما هر عضو `frontier` سه بخش دارد: حالت فعلی، مسیر طی‌شده تا آن حالت و عمق فعلی:

```
1 frontier = [(initial_state, [], 0)]
```

در اینجا `frontier` یک لیست پایتونی است و با `pop` از انتهای آن برمی‌داریم. بنابراین رفتار آن پشته یا LIFO است. برخلاف BFS که مسیر را فقط در پایان با `came_from` بازسازی می‌کرد، در این پیاده‌سازی مسیر همراه هر حالت نگه داشته می‌شود. دلیلش سادگی پیاده‌سازی است: وقتی به یک جانشین می‌رویم، مسیر جدید برابر مسیر قبلی به اضافه عمل جدید است. در ابتدای فایل دو مقدار قراردادی تعریف شده‌اند:

```
1 CUTOFF = "cutoff"
2 FAILURE = "failure"
```

CUTOFF یعنی جستجو به خاطر حد عمق متوقف شده و ممکن است اگر حد را بیشتر کنیم جواب پیدا شود. FAILURE یعنی در محدوده بررسی‌شده جوابی پیدا نشده است. این تفاوت برای حالت IDS مهم است، چون اگر نتیجه CUTOFF باشد باید حد عمق را افزایش دهیم، اما اگر FAILURE باشد یعنی ادامه دادن فایده‌ای ندارد. تابع `_state_key` برای تشخیص وضعیت‌های تکراری استفاده می‌شود:

```

1 return (
2     state.get_agent_position(),
3     frozenset(state.get_targets_positions()),
4     state.get_enemy_cycle(),
5     state.has_weapon(),
6     state.is_enemy_alive(),
7 )

```

در این کلید، موقعیت عامل، هدف‌های باقی‌مانده، چرخه دشمن، وضعیت سلاح و زنده بودن دشمن ذخیره می‌شود. خود شماره گام به طور مستقیم در کلید نیامده است، چون برای برخورد با دشمن فقط چرخه حرکت دشمن مهم است. هزینه مسیر هم در کلید نیست، چون DLS بر اساس عمق و تعداد حرکت کار می‌کند، نه بر اساس هزینه. در تابع `depth_limited_search` دیکشنری `best_depth` کمترین عمقی را نگه می‌دارد که هر وضعیت مهم در آن دیده شده است:

```

1 best_depth = {start_key: 0}

```

اگر بعداً همان وضعیت با عمق بیشتر یا برابر دیده شود، آن مسیر کنار گذاشته می‌شود، چون رسیدن به همان وضعیت با حرکت‌های بیشتر برای جستجوی عمقی کمکی نمی‌کند. حلقه اصلی با برداشتن آخرین عضو پشته شروع می‌شود:

```

1 state, actions, depth = frontier.pop()

```

بعد از آن، اگر حالت هدف باشد، همان مسیر `actions` برگردانده می‌شود. اگر هدف نباشد و عمق حالت از حد مجاز بیشتر شده باشد، نتیجه `CUTOFF` می‌شود و آن شاخه دیگر ادامه پیدا نمی‌کند:

```

1 if depth > limit:
2     result = CUTOFF
3     continue

```

سپس جانشین‌ها بررسی می‌شوند. در کد از `reversed(state.get_successors())` استفاده شده است:

```

1 for action, _, next_state in reversed(state.get_successors()):
2     ...

```

دلیل `reversed` این است که پشته آخرین عضو اضافه‌شده را زودتر بیرون می‌آورد. ترتیب اصلی حرکت‌ها در `DIRECTIONS` برابر `U, D, L, R` است. اگر جانشین‌ها را برعکس اضافه کنیم، هنگام `pop` شدن دوباره همان ترتیب طبیعی‌تر `U` قبل از `D` و سپس `L` و `R` بررسی می‌شود. مثل `BFS`، در `DLS` هم هزینه حرکت با `_` نادیده گرفته شده است. دلیلش این است که این الگوریتم مسیر را بر اساس عمق محدود می‌کند، نه بر اساس هزینه واقعی. حالت‌های برخوردی حذف می‌شوند:

```

1 if next_state.is_collision_state():
2     continue

```

سپس کلید حالت بعدی و عمق جدید ساخته می‌شود. اگر همان کلید قبلاً با عمق کمتر یا مساوی دیده شده باشد، از آن صرف‌نظر می‌کنیم:

```

1 if new_depth >= best_depth.get(key, float("inf")):
2     continue

```

در غیر این صورت، عمق بهتر ثبت می‌شود و حالت بعدی همراه با مسیر جدید وارد پشته می‌شود:

```

1 best_depth[key] = new_depth
2 frontier.append((next_state, actions + [action], new_depth))

```

وقتی مقدار `limit` به تابع اصلی داده نشود، الگوریتم حد عمق را از صفر تا `rows * cols` زیاد می‌کند؛ بنابراین رفتار آن شبیه `Iterative Deepening Search` است. مزیت این روش این است که حافظه زیادی مثل `BFS` مصرف نمی‌کند، چون بیشتر با پشته مسیرهای فعلی کار می‌کند.

در حالت `IDS` اگر یک حد عمق به `CUTOFF` برسد، حد بعدی امتحان می‌شود. اگر نتیجه `FAILURE` باشد، تابع لیست خالی برمی‌گرداند. این طراحی باعث می‌شود وقتی عمق جواب را از قبل نمی‌دانیم، به جای انتخاب یک حد ثابت، حد را مرحله‌به‌مرحله زیاد کنیم.

ضعف `DLS/IDS` این است که با زیاد شدن حد عمق، بخش‌هایی از جستجو دوباره تکرار می‌شوند. به همین دلیل تعداد بسط‌ها معمولاً از `BFS` بیشتر می‌شود. همچنین این الگوریتم مثل `BFS` هزینه واقعی حرکت‌ها را به عنوان اولویت اصلی در نظر نمی‌گیرد و بیشتر بر تعداد حرکت‌ها تکیه دارد.

۱۱ الگوریتم UCS

UCS در ucs.py با PriorityQueue پیاده‌سازی شده است. این الگوریتم شبیه BFS یک frontier دارد، اما به جای اینکه حالت‌ها را بر اساس تعداد حرکت بیرون بیاورد، همیشه حالتی را انتخاب می‌کند که کمترین هزینه واقعی طی‌شده از ابتدا تا آن حالت را دارد. بنابراین برای این پروژه که خانه عادی هزینه 5 و خانه یخی هزینه 100 دارد، UCS مناسب‌تر از BFS است. در شروع، صف اولویت، شمارنده، دیکشنری مسیر و دیکشنری هزینه ساخته می‌شوند:

```
1 frontier = PriorityQueue()
2 counter = itertools.count()
3 frontier.put((0, 0, next(counter), initial_state))
4 came_from = {initial_state: (None, None)}
5 cost_so_far = {initial_state: 0}
```

هر عضو صف چهار بخش دارد:

```
1 (priority, new_cost, counter, state)
```

priority معیار اصلی مرتب‌سازی صف است و در UCS همان هزینه واقعی مسیر است:

```
1 priority = new_cost
```

new_cost هم جداگانه در صف ذخیره شده است تا وقتی حالت از صف بیرون آمد، بتوانیم تشخیص دهیم این نسخه هنوز بهترین نسخه است یا قدیمی شده است. counter برای شکستن تساوی استفاده می‌شود. اگر دو حالت هزینه یکسان داشته باشند، PriorityQueue به عضو بعدی نگاه می‌کند. بدون counter ممکن است صف مجبور شود خود GameState ها را با هم مقایسه کند، در حالی که مقایسه ترتیبی برای آن‌ها تعریف نشده است. دیکشنری cost_so_far بهترین هزینه شناخته‌شده برای رسیدن به هر حالت را نگه می‌دارد. دیکشنری came_from نیز برای بازسازی مسیر نهایی است. در حلقه اصلی، کم‌هزینه‌ترین حالت از صف خارج می‌شود:

```
1 _, popped_cost, _, current_state = frontier.get()
```

در صف اولویت ممکن است نسخه قدیمی یک حالت باقی بماند. برای حل این مشکل، هزینه‌ای که همراه حالت از صف بیرون آمده با مقدار فعلی cost_so_far مقایسه می‌شود. اگر برابر نباشد، آن ورودی قدیمی است و بسط داده نمی‌شود. این کار با نام stale-entry skip شناخته می‌شود.

```
1 if popped_cost != cost_so_far[current_state]:
2     continue
```

بعد از حذف ورودی‌های قدیمی، اگر حالت هدف باشد، جستجو متوقف می‌شود. در UCS این کار درست است، چون همه هزینه‌های حرکت مثبت هستند و صف همیشه کم‌هزینه‌ترین حالت موجود را خارج می‌کند. بنابراین اولین هدفی که از صف بیرون می‌آید، ارزان‌ترین مسیر پیدا شده به هدف است. اگر حالت هدف نباشد، جانشین‌های آن ساخته می‌شوند:

```
1 for action, step_cost, next_state in current_state.get_successors():
2     new_cost = cost_so_far[current_state] + step_cost
```

اینجا برخلاف BFS و DLS، مقدار step_cost استفاده می‌شود. هزینه جدید برابر هزینه رسیدن به حالت فعلی به اضافه هزینه حرکت تازه است. اگر حالت بعدی قبلاً دیده نشده باشد، یا مسیر تازه هزینه کمتری نسبت به بهترین مسیر قبلی داشته باشد، اطلاعات آن به‌روزرسانی می‌شود:

```
1 if (
2     next_state not in came_from or new_cost < cost_so_far[next_state]
3 ) and not next_state.is_collision_state():
4     cost_so_far[next_state] = new_cost
5     priority = new_cost
6     frontier.put((priority, new_cost, next(counter), next_state))
7     came_from[next_state] = (current_state, action)
```

شرط مربوط به collision مثل الگوریتم‌های دیگر مسیرهای برخوردی را حذف می‌کند. بخش مقایسه هزینه هم تفاوت اصلی UCS با BFS است: در BFS اولین رسیدن به یک حالت کافی است، چون عمق کمتر مهم است؛ اما در UCS ممکن است همان حالت بعداً با مسیر طولانی‌تر ولی ارزان‌تر پیدا شود، پس باید اجازه به‌روزرسانی داشته باشیم.

در پایان، اگر هدف پیدا شده باشد، مسیر مثل BFS از goal_state به عقب بازسازی می‌شود و سپس معکوس می‌گردد. اگر هدفی پیدا نشده باشد، لیست خالی برگردانده می‌شود.

تفاوت اصلی UCS با BFS این است که کوتاه‌ترین مسیر از نظر تعداد حرکت لزوماً اولویت ندارد؛ مسیری جلوتر می‌آید که تا آن لحظه هزینه کمتری داشته باشد. ضعف UCS این است که هیچ تخمینی از فاصله تا هدف ندارد. بنابراین مخصوصاً در نقشه‌هایی با هدف‌های زیاد، حالت‌های بسیار زیادی را بررسی می‌کند.

۱۲ الگوریتم A^*

در `a_star.py` مهم‌ترین بخش کار، طراحی A^* بود. در این الگوریتم، اولویت هر حالت ترکیبی از هزینه واقعی طی شده و تخمین هزینه باقی‌مانده است:

$$f(n) = g(n) + h(n)$$

$g(n)$ همان مقدار `cost_so_far` است و $h(n)$ تابع اکتشافی است. هرچه $h(n)$ به هزینه واقعی باقی‌مانده نزدیک‌تر باشد، A^* به جای شبیه شدن به UCS، مستقیم‌تر به سمت جواب می‌رود و گره‌های کمتری را بسط می‌دهد. ساختار کلی صف در نسخه نهایی به شکل زیر است:

```
1 priority = new_cost + h
2 frontier.put((
3     priority,
4     weapon_awareness_key(next_state, cache),
5     h,
6     new_cost,
7     next(counter),
8     next_state
9 ))
```

عضو اول همان $f(n)$ است و معیار اصلی انتخاب حالت بعدی محسوب می‌شود. مقدار `new_cost` نیز در صف ذخیره شده است تا ورودی‌های قدیمی حذف شوند:

```
1 if popped_cost != cost_so_far[current_state]:
2     continue
```

وجود `counter` برای این است که اگر چند حالت اولویت‌های یکسان داشتند، صف اولویت مجبور نشود خود `GameState` ها را با هم مقایسه کند.

۱۳ تابع اکتشافی A^*

در نسخه نهایی، تابع اکتشافی را از چند پایین‌کران ساده اما مؤثر ساختیم. هدف این بود که تابع بیش از حد پیچیده نشود، اما نسبت به فاصله منتهن اطلاعات بیشتری از نقشه داشته باشد؛ یعنی دیوارها، خانه‌های یخی، چندهدفه بودن مسئله و وجود سلاح را در نظر بگیرد.

۱.۱۳ استفاده از فاصله `Dijkstra`

فاصله منتهن فقط اختلاف سطر و ستون را حساب می‌کند. این فاصله نه دیوار را می‌بیند، نه خانه یخی را، و نه مسیرهای بسته را. در نقشه‌های این پروژه چنین تخمینی خیلی خوش‌بینانه است و باعث می‌شود A^* گره‌های زیادی را مثل UCS بررسی کند. به همین دلیل فاصله بین دو خانه با `Dijkstra` روی خود نقشه حساب می‌شود. در این محاسبه، رأس‌ها خانه‌های قابل عبور نقشه هستند، خانه‌های `R` نادیده گرفته می‌شوند، و هزینه ورود به خانه بعدی با `get_terrain_cost` به دست می‌آید. بنابراین فاصله محاسبه‌شده همان نوع هزینه‌ای است که الگوریتم اصلی هم پرداخت می‌کند. فرمول به‌روزرسانی فاصله در این `Dijkstra` به شکل زیر است:

$$dist(v) = \min(dist(v), dist(u) + cost(v))$$

در این فرمول `u` خانه فعلی، `v` خانه بعدی و `cost(v)` هزینه ورود به خانه `v` است. چون همه هزینه‌های حرکت مثبت هستند، `Dijkstra` برای پیدا کردن کوتاه‌ترین هزینه مکانی مناسب است. دلیل استفاده از این فاصله به جای منتهن این است که منتهن ممکن است از روی دیوار یا خانه‌های یخی عبور خیالی کند، اما `Dijkstra` همان محدودیت‌های نقشه را رعایت می‌کند. نکته مهم این است که `Dijkstra` اینجا روی مختصات‌ها اجرا می‌شود، نه روی `GameState` کامل. اگر برای محاسبه فاصله از `get_successors` حالت کامل استفاده می‌کردیم، هدف‌های جمع‌شده، زمان دشمن و وضعیت سلاح هم وارد محاسبه می‌شدند و خود تابع اکتشافی تبدیل به یک جستجوی بزرگ می‌شد. نسخه نهایی فقط کوتاه‌ترین فاصله مکانی روی نقشه را می‌خواهد.

۲.۱۳ کش کردن فاصله‌ها

تابع اکتشافی بارها فاصله بین خانه‌های تکراری را لازم دارد؛ برای مثال فاصله بین دو هدف ممکن است در حالت‌های زیادی دوباره خواسته شود. اجرای دوباره `Dijkstra` برای هر درخواست، زمان اجرا را زیاد می‌کند. برای همین از `cache` استفاده شده است:

```
1 (source_pos, target_pos) -> min_distance
```

کلید کش جهت دار است؛ یعنی فاصله A به B جدا از فاصله B به A ذخیره می شود. دلیلش این است که هزینه حرکت هنگام ورود به خانه مقصد پرداخت می شود. پس اگر یکی از دو خانه یخی باشد، رفتن از A به B ممکن است دقیقاً با برگشت از B به A برابر نباشد. کش مقدار تابع اکتشافی را تغییر نمی دهد؛ فقط همان محاسبه را سریع تر می کند. در آزمایش ها، بعد از اضافه شدن کش، زمان اجرای نقشه medium از چند ثانیه به کمتر از یک ثانیه رسید. بنابراین کش برای کاهش زمان اجراست، نه برای کاهش مستقیم تعداد گره های بسط داده شده.

۳.۱۳ پایین کران نزدیک ترین هدف و MST

اگر فقط فاصله تا نزدیک ترین هدف را حساب کنیم، تابع اکتشافی فقط اولین هدف را می بیند و بقیه کار را فراموش می کند. برای حل این مشکل، هزینه تخمینی از دو بخش ساخته شده است:

$$h_{mst}(s, T) = \min_{t \in T} d(s, t) + MST(T)$$

در این فرمول موقعیت فعلی عامل، T مجموعه هدف های باقی مانده و $d(x, y)$ کوتاه ترین هزینه مکانی بین دو خانه با Dijkstra است.

بخش اول می گوید عامل حداقل باید به یکی از هدف های باقی مانده برسد. بخش دوم هزینه وصل کردن همه هدف های باقی مانده به هم است.

MST یا درخت پوشای کمینه، ارزان ترین راه برای متصل کردن همه هدف ها است. هر مسیر واقعی که همه هدف ها را جمع می کند، در نهایت این هدف ها را به شکلی به هم مرتبط کرده است. بنابراین هزینه MST روی فاصله های واقعی نقشه یک پایین کران معتبر برای کار باقی مانده است. به بیان ساده، مسیر واقعی برای جمع آوری همه هدف ها یک مسیر پیوسته بین هدف ها می سازد؛ اگر یال های اضافه و ترتیب حرکت را کنار بگذاریم، هنوز یک اتصال بین هدف ها باقی می ماند. هزینه ارزان ترین اتصال ممکن همان MST است، پس از هزینه مسیر واقعی بیشتر نمی شود.

تابع mst_cost با ایده الگوریتم پریم نوشته شده است. ابتدا یک هدف به عنوان ریشه انتخاب می شود. سپس برای هر هدف دیده نشده، بهترین فاصله فعلی تا درخت نگه داشته می شود. در هر مرحله هدفی که ارزان تر به درخت وصل می شود انتخاب شده و مقدارهای دیگر به روزرسانی می شوند. این نسخه نسبت به بررسی دوباره همه یال ها در هر مرحله، محاسبات تکراری کمتری دارد. به صورت خلاصه، در هر مرحله داریم:

$$best(v) = \min_{u \in visited} d(u, v)$$

و هدفی انتخاب می شود که مقدار $best(v)$ آن کمترین باشد.

۴.۱۳ پایین کران سه تایی هدف ها

MST هنوز یک ضعف دارد: درخت می تواند شاخه داشته باشد، ولی عامل نمی تواند شاخه شاخه حرکت کند. عامل باید یک مسیر واقعی را طی کند و هدف ها را یکی یکی جمع کند. برای نقشه هایی که هدف های زیادی دارند، مثل medium، همین تفاوت باعث می شود MST گاهی تخمین را خیلی پایین بدهد.

برای قوی تر شدن تخمین بدون وارد کردن برنامه ریزی پویای سنگین، تابع $target_triple_lower_bound$ اضافه شد. این تابع همه گروه های سه تایی از هدف های باقی مانده را بررسی می کند. برای هر سه هدف، هر شش ترتیب ممکن بازدید امتحان می شود:

```
1 start -> first -> second -> third
```

برای هر سه تایی، ارزان ترین ترتیب انتخاب می شود. سپس بین همه سه تایی ها، سخت ترین سه تایی نگه داشته می شود. دلیل معتبر بودن این پایین کران این است که مسیر نهایی باید همه هدف ها را جمع کند؛ پس حتماً باید هر سه تایی دلخواه از هدف ها را هم در یک ترتیبی بازدید کرده باشد. اگر حتی بازدید از یک سه تایی خاص حداقل هزینه زیادی لازم داشته باشد، کل مسیر هم نمی تواند از آن کمتر باشد. فرمول این بخش به شکل زیر است:

$$h_3(s, T) = \max_{\substack{Q \subseteq T \\ |Q|=3}} \min_{(a,b,c) \in Perm(Q)} (d(s, a) + d(a, b) + d(b, c))$$

در این فرمول، Q یک گروه سه تایی از هدف ها و $Perm(Q)$ همه ترتیب های ممکن همان سه هدف است. ابتدا برای هر سه تایی ارزان ترین ترتیب انتخاب می شود، سپس بین سه تایی ها بیشترین مقدار گرفته می شود؛ چون سخت ترین سه تایی، الزام قوی تری برای مسیر نهایی ایجاد می کند.

در نهایت این مقدار با تخمین قبلی جمع نمی شود، چون ممکن است باعث دوباره شماری هزینه شود. به جای آن از max استفاده می شود:

```
1 return max(weapon_aware_cost, triple_cost)
```

یعنی هر کدام از دو پایین کران قوی تر بود، همان به عنوان تخمین استفاده می شود. همین بخش تعداد بسط های نقشه medium را از حدود 692 به 394 کاهش داد.

۵.۱۳ در نظر گرفتن سلاح

در بعضی نقشه‌ها سلاح وجود دارد. اگر عامل سلاح را بگیرد، برخورد با دشمن شکست محسوب نمی‌شود و حتی ممکن است دشمن حذف شود. تابع اکتشافی دو حالت را مقایسه می‌کند:

- مستقیم به سمت جمع‌آوری هدف‌ها رفتن.
 - ابتدا رفتن به خانه سلاح و سپس تخمین هزینه جمع‌آوری هدف‌ها از موقعیت سلاح.
- کد این ایده به صورت خلاصه چنین است:

$$h_{\text{weapon}}(s, T) = \min(h_{\text{targets}}(s, T), d(s, w) + h_{\text{targets}}(w, T))$$

در این فرمول w موقعیت سلاح است و h_{targets} همان تخمین نزدیک‌ترین هدف به علاوه MST است. اگر سلاح وجود نداشته باشد، یا عامل قبلاً سلاح را گرفته باشد، همان $h_{\text{targets}}(s, T)$ استفاده می‌شود.

استفاده از $\min$ یعنی تابع اکتشافی عامل را مجبور نمی‌کند حتماً سلاح را بردارد. اگر رفتن به سلاح در تخمین مفید نباشد، همان مسیر مستقیم در نظر گرفته می‌شود. در نسخه نهایی، مقدار نهایی تابع اکتشافی چنین ساخته می‌شود:

$$h(s, T) = \max(h_{\text{weapon}}(s, T), h_3(s, T))$$

دلیل استفاده از $\max$ این است که هر دو مقدار، تخمین‌هایی از کار باقی‌مانده هستند. جمع کردن آن‌ها ممکن است یک بخش از مسیر را دوبار حساب کند، اما گرفتن بیشینه فقط قوی‌ترین تخمین را انتخاب می‌کند.

۶.۱۳ Tiebreakers

گاهی چند حالت مقدار $f(n)$ یکسانی دارند. در این حالت، ترتیب اعضای بعدی صف مشخص می‌کند کدام حالت زودتر بررسی شود. به این مقدارها tie-breaker می‌گوییم. تای‌بریکرها هزینه مسیر را تغییر نمی‌دهند و فقط بین حالت‌هایی که از نظر اولویت اصلی برابرند، یک ترتیب ترجیحی می‌سازند.

در نسخه نهایی، بعد از priority_key ابتدا $\text{weapon_awareness_key}$ قرار دارد. اگر عامل هنوز سلاح ندارد و دشمن زنده است، حالت‌هایی که سلاح را گرفته‌اند یا به سلاح نزدیک‌ترند زودتر بررسی می‌شوند. بعد از آن مقدار h قرار دارد؛ یعنی اگر دو حالت f برابر داشته باشند، حالتی که تخمین باقی‌مانده کمتری دارد زودتر انتخاب می‌شود. این کار معمولاً جستجو را به سمت حالت‌هایی می‌برد که به تکمیل مسیر نزدیک‌تر هستند. پس ترتیب صف به صورت زیر معنا می‌شود:

```
1 (f, weapon_key, h, g, counter, state)
```

ابتدا کمترین f انتخاب می‌شود. اگر برابر بود، کلید سلاح، سپس مقدار h ، سپس هزینه واقعی g و در نهایت counter تصمیم می‌گیرند. تای‌بریکرها مقدار f را تغییر نمی‌دهند؛ فقط وقتی چند حالت از نظر معیار اصلی برابرند، ترتیب بررسی را مشخص می‌کنند.

۱۴ امتیازدهی

در `game.py` کلاس `ScoreCalculator` امتیاز نهایی را محاسبه می‌کند. اگر همه هدف‌ها جمع شده باشند، به تعداد هدف‌ها پاداش داده می‌شود. اگر برخورد شکست‌آمیز رخ دهد، جریمه بزرگ کم می‌شود. اگر دشمن حذف شده باشد، پاداش اضافه داده می‌شود. در پایان هزینه کل مسیر از امتیاز کم می‌شود:

```
1 if state.is_goal_state():
2     score += self.target_count * TARGET_REWARD
3 if state.is_collision_state():
4     score -= ENEMY_COLLISION_PENALTY
5 if not state.is_enemy_alive():
6     score += ENEMY_KILL_REWARD
7 score -= total_cost
```

پس هدف فقط کم کردن تعداد حرکت نیست؛ مسیر بهتر مسیری است که با هزینه کمتری هدف‌ها را جمع کند و در صورت مفید بودن، از سلاح هم استفاده کند.

۱۵ نتایج اجرا

برای اندازه‌گیری، الگوریتم‌ها بدون اجرای پنجره گرافیکی و با استفاده از `CountingGameState` اجرا شدند. در اجراهای سنگین، اگر الگوریتم در ۱۰ ثانیه به جواب نمی‌رسید، نتیجه به صورت `timeout` ثبت شد. جدول زیر مقایسه الگوریتم‌ها روی چند نقشه مهم را نشان می‌دهد:

نقشه	الگوریتم	وضعیت	تعداد حرکت	هزینه مسیر	گره‌های بسط داده شده
easy-no-weapon	BFS	OK	۱۸	۲۸۰	۸۸۸
easy-no-weapon	DLS	OK	۱۸	۲۸۰	۲۶۵۳
easy-no-weapon	UCS	OK	۲۲	۱۱۰	۴۸۶
easy-no-weapon	A*	OK	۲۲	۱۱۰	۴۰
easy-weapon	BFS	OK	۱۶	۲۷۰	۶۵۱
easy-weapon	DLS	OK	۱۶	۲۷۰	۱۵۸۶
easy-weapon	UCS	OK	۱۸	۹۰	۴۱۷
easy-weapon	A*	OK	۱۸	۹۰	۱۸
medium-no-weapon	BFS	OK	۲۸	۳۳۰	۸۷۷۸
medium-no-weapon	DLS	OK	۲۸	۳۳۰	۴۰۲۳۷
medium-no-weapon	UCS	OK	۳۴	۱۷۰	۴۷۸۹
medium-no-weapon	A*	OK	۳۴	۱۷۰	۵۰
medium-weapon	BFS	OK	۲۶	۲۲۵	۶۵۰۶۹
medium-weapon	DLS	OK	۲۶	۲۲۵	۲۹۱۸۶۳
medium-weapon	UCS	OK	۲۶	۱۳۰	۴۰۰۶۷
medium-weapon	A*	OK	۲۶	۱۳۰	۲۶
medium	BFS	timeout	-	-	۷۴۶۳۸۹
medium	DLS	timeout	-	-	۶۸۷۲۰۲
medium	UCS	OK	۴۶	۴۲۰	۳۲۰۷۲۱
medium	A*	OK	۴۶	۴۲۰	۳۹۴
hard-no-weapon	BFS	OK	۴۱	۶۸۰	۷۰۵۰
hard-no-weapon	DLS	OK	۴۱	۶۸۰	۵۳۰۵۴
hard-no-weapon	UCS	OK	۴۶	۳۲۵	۹۲۸۲
hard-no-weapon	A*	OK	۴۶	۳۲۵	۲۱۱

این نتایج نشان می‌دهد که A* همان هزینه بهینه UCS را پیدا می‌کند، اما با تعداد بسط بسیار کمتر. برای مثال در easy-no-weapon تعداد بسط از 486 در UCS به 40 در A* رسیده است، یعنی شرط کمتر از ۱۰۰ گره برای نقشه ساده رعایت شده است. در نقشه medium نیز A* هزینه مسیر UCS را ننگه داشت، اما تعداد بسط را از 320721 به 394 رساند و شرط کمتر از ۵۰۰ گره را رعایت کرد. در نقشه hard-no-weapon نیز BFS و DLS مسیر ۴۱ حرکتی پیدا کردند، اما هزینه آن 680 بود. در مقابل، UCS و A* مسیر ۴۶ حرکتی با هزینه کمتر 325 پیدا کردند و A* تعداد بسط را از 9282 در UCS به 211 کاهش داد. نتایج A* روی همه نقشه‌های موجود نیز به شکل زیر بود. ستون‌های سلاح و حذف دشمن نشان می‌دهند در کدام گام سلاح گرفته شده یا دشمن حذف شده است:

نقشه	تعداد حرکت	هزینه مسیر	گره‌های بسط داده شده	گام سلاح	گام حذف دشمن
easy-no-weapon	۲۲	۱۱۰	۴۰	-	-
easy-weapon	۱۸	۹۰	۱۸	۱۰	۱۱
medium-no-weapon	۳۴	۱۷۰	۵۰	-	-
medium-no-weapon2	۳۰	۱۵۰	۴۹	-	-
medium-weapon	۲۶	۱۳۰	۲۶	۳	۱۶
medium	۴۶	۴۲۰	۳۹۴	-	-
hard-no-weapon	۴۶	۳۲۵	۲۱۱	-	-
hard-no-weapon2	۴۳	۲۱۵	۹۹	-	-
hard-weapon	۴۲	۳۰۵	۹۲	۲۰	۲۴
extra-no-weapon	۵۸	۳۸۵	۲۰۵	-	-
extra-weapon	۵۴	۳۶۵	۲۰۴	۱۵	۴۵

۱۶ جمع‌بندی

در این پروژه مسئله به صورت یک جستجوی گراف‌ی مدل شد. حالت شامل موقعیت عامل، هدف‌های باقی‌مانده، زمان، وضعیت دشمن و وضعیت سلاح است. الگوریتم BFS ساده است ولی به دلیل هزینه‌های متفاوت، تضمین کم‌هزینه‌ترین مسیر را ندارد. DLS/IDS از نظر حافظه بهتر است، اما بسط‌های تکراری زیادی انجام می‌دهد. UCS بهینه است، ولی چون آگاهی از هدف ندارد تعداد زیادی گره را بررسی می‌کند.

بهترین نتیجه با A* به دست آمد. تابع اکتشافی آن از فاصله‌های واقعی Dijkstra روی نقشه، کش فاصله‌ها، MST بین هدف‌های باقی‌مانده، پایین‌کران سه‌تایی هدف‌ها و آگاهی از سلاح استفاده می‌کند. این ترکیب پیچیدگی کد را در حد قابل توضیح نگه می‌دارد، اما تعداد بسط‌ها را در نقشه‌های ساده، متوسط و سخت از حد‌های امتیاز کامل کمتر می‌کند.

مراجع

[۱] [https://www.Amit.J.Patel, "Introduction to the A* Algorithm", Red Blob Games, 2014.](https://www.Amit.J.Patel,) (accessed 2026-05-24). redblobgames.com/pathfinding/a-star/introduction.html